Phase 1: Database Architecture & Foundation

The first step was moving away from disconnected CSV files and building a **Relational Database** in PostgreSQL.

- **Relational Schema Design:** We built 9 interconnected tables. Instead of keeping data flat, we established **Primary and Foreign Keys** to link orders to customers, products, and sellers.
- **Column Optimization:** All technical and long column names were renamed to professional, shorthand versions (e.g., order_delivered_customer_date became delivered_at). This makes SQL queries easier to write and Power BI fields cleaner.
- **Logical ID Separation:** We correctly identified the difference between cust_id (transactional) and unique_id (customer loyalty), setting the foundation for both logistics and marketing analysis.

Phase 2: Data Engineering & ETL (Cleaning)

This was the most critical phase, where we solved real-world data quality issues that would have broken a standard dashboard.

- **ETL Loading Strategy:** We established a **"Golden Load Order"** (Parent tables before Child tables) to satisfy referential integrity and avoid Foreign Key errors during import.
- **Metadata Patching:** We identified that the Brazilian translation file was incomplete. We performed a **manual data patch** to add missing categories like pc_gamer so the product import wouldn't fail.
- **Deduplication Strategy:** To handle duplicate rev_id values in the reviews table, we implemented a **Staging Table** approach, using DISTINCT ON logic to ensure only unique, clean records entered the final database.
- **Encoding & Permissions:** We bypassed server-level "Permission Denied" errors and handled Brazilian special characters (UTF8) to ensure city names like *São Paulo* remained accurate.

Phase 3: Advanced Analytical Modeling (The Views)

Instead of feeding messy raw tables into Power BI, we built **4 Master Views** that perform the heavy lifting. This is where the **Supply Chain Domain Knowledge** was applied.

1. **Logistics Performance View:** This tracks the **Order Life Cycle**. It breaks down the wait time into three stages:
    - **Warehouse Time:** From purchase to carrier handoff.
    - **Shipping Time:** From carrier handoff to the customer's door.
    - **SLA Gap:** Calculating exactly how many days early or late a delivery was compared to the estimate.

2. **Product Master View:** This view creates a "Clean Product Catalog." It joins **English translations** and calculates **Volumetric Weight** ($L \times W \times H / 6000$), a vital metric for shipping cost analysis.
3. **Geospatial Distance View:** By averaging 1M+ rows of coordinates into a clean zip lookup, we connected **Seller Lat/Long** to **Customer Lat/Long**. This allows for "Route Analysis" and accurate mapping.
4. **Customer Feedback View:** This links logistics speed to **Review Scores**, allowing us to prove exactly how much a delivery delay hurts the brand's rating.

The Resulting Data Logic

The database is no longer just a collection of files; it is a **Supply Chain Model**. It is designed to answer the three big questions:

- **Who is responsible for delays?** (The Seller or the Carrier).
- **Where is the money going?** (Freight costs vs. Product size and distance).
- **How does it impact the customer?** (Review score correlation).

Current Project Status

- **SQL Phase:** Complete. The database is clean, joined, and calculated.
- **Data Health:** Verified via "Handshake Query" ensuring all views communicate via ord_id.
- **Next Phase:** Visualizing insights in **Power BI**.

Phase 4: Visual Intelligence & Dashboard Design

With the data engine running in PostgreSQL, we transitioned to Power BI to turn raw metrics into a decision-making tool.

- **The Star Schema Architecture:** We implemented a **Star Schema** in Power BI, placing the Logistics Performance View at the center (Fact Table). By connecting our Product, Feedback, and Geo views (Dimension Tables) via ord_id, we ensured that any filter applied to one chart instantly updates the entire dashboard without data "leakage" or double-counting.
- **Custom Business Logic (DAX):** We moved beyond standard counts by developing custom DAX (Data Analysis Expressions). Our primary metric, **On-Time Delivery Rate %**, uses advanced filtering logic ($DIVIDE$ and $CALCULATE$) to provide a dynamic Service Level Agreement (SLA) percentage that responds to regional and category slicers.

Phase 5: Key Business Insights & Findings

Based on the final dashboard, the following strategic insights were uncovered:

Insight Category	Discovery	Strategic Recommendation
Logistics Bottlenecks	"Warehouse Time" accounts for 25% of total delays in specific high-volume states.	Implement stricter "Sourced-to-Shipped" deadlines for sellers in those regions.
Geospatial Efficiency	High freight costs in Northern Brazil are directly correlated with "Volumetric Weight" rather than distance.	Renegotiate carrier contracts for bulkier items or incentivize local warehousing.
SLA Impact	Average Review Scores drop by 1.2 points for every 3 days a package is delayed beyond the estimate.	Proactively issue "Delay Vouchers" when the shipping time exceeds 48 hours past the SLA.

Phase 6: Technical Challenges & Troubleshooting (The "War Room" Logs)

Building a data pipeline is never smooth. Below are the four major technical hurdles we faced and the engineering workarounds implemented to solve them:

1. The "SASL/SCRAM" Security Wall (SQL to Power BI Connection)

- **The Problem:** Power BI refused to connect to the PostgreSQL server, throwing a "SASL/SCRAM-SHA-256" authentication error. This happened because the database's modern security protocols were "too high" for the standard Power BI driver to bypass without a specific password handshake.
- **The Solution:** We performed a **Security Level Downgrade** for the local environment. By accessing the pg_hba.conf system file as an Administrator, we modified the authentication method from scram-sha-256 to trust for local IPv4 and IPv6 addresses. This "unlocked the door," allowing Power BI to communicate directly with the data engine.

2. The "No Such Host" & Loopback Issue

- **The Problem:** Power BI could not find the database using the standard "localhost" name, resulting in a "No such host is known" error. This was likely due to a DNS conflict in the Windows network settings.
- **The Solution:** We bypassed the "nickname" system and used the **Hardcoded Loopback IP Address (127.0.0.1)**. This forced Power BI to look directly at the internal hardware port (5432) where PostgreSQL was listening, eliminating any naming confusion.

3. The "Many-to-Many" Relationship Trap

- **The Problem:** Because the Olist dataset contains multiple items per order, Power BI initially flagged the relationships between tables as "Many-to-Many." This threatened to "double-count" revenue and lead times, which would lead to fake, inflated numbers.

- **The Solution:** We implemented a **Bi-Directional Filtering Strategy** combined with "Single" cross-filters for the map. By setting the Product and Feedback tables to "Both" directions, we allowed filters to travel through the Logistics View (The Fact Table) while maintaining data integrity. We verified the results by checking the "Distinct Count" of orders against our original SQL counts (96,469 unique orders).

4. Geospatial "Data Drift" (The US Bubbles)

- **The Problem:** The map visual initially displayed bubbles in the United States and Europe instead of Brazil. This happened because Power BI's Bing Map engine mistook Brazilian state codes (like "SP") for other international locations.
- **The Solution:** We implemented **Data Categorization**. We manually assigned "State/Province" and "City" categories to the columns in Power BI. To ensure 100% accuracy, we eventually switched to using the **Latitude and Longitude coordinates** we pre-processed in SQL, forcing every bubble to its exact physical location in Brazil